Indexing for Concurrency-Aware Distributed Joins

Soubhik Gon
Hitanshu Satpathy
Amit Bhagat

IIIT Bhubaneswar
Department of Computer Science and Engineering

8th Semester Mid-Sem Progress Presentation

Problem Context

- Rapidly growing data volumes in modern database systems demand highly efficient data access and query processing mechanisms
- Traditional B+ tree indexes struggle to scale under complex join workloads and large-scale data filtering operations
- Data skew leads to uneven data distribution, causing load imbalance and performance degradation in parallel database systems

Problem Context

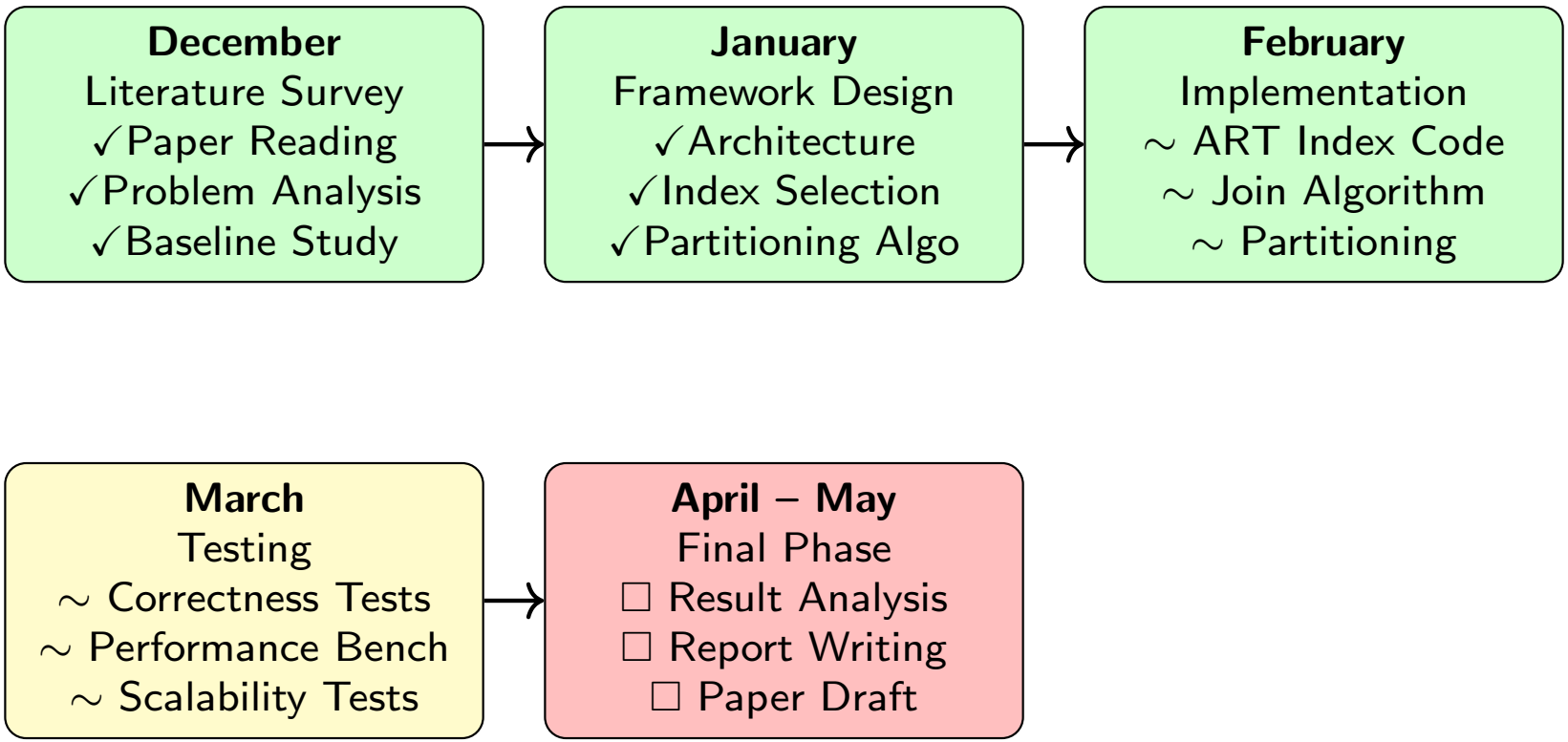
- Lock contention emerges as a major bottleneck in concurrent transaction processing, reducing throughput and increasing response latency
- Parallel join execution faces significant challenges due to data skew, inter-operator data redistribution, and synchronization overheads
- Concurrent index operations further exacerbate performance issues because of lock contention and cache coherency effects

Project Focus: Concurrency-Aware Indexing

- Analyze the limitations of traditional index structures in modern distributed database environments
- Employ lock-free and latch-free index structures such as BW-Tree, ART and Masstree to reduce contention
- Incorporate skew-resistant partitioning to mitigate load imbalance in parallel join processing
- Explore hardware-aware execution techniques to exploit CPU-level optimizations and improve throughput

Implementation Git Repository:
https://github.com/zakhaev26/db-research-paper

Project Timeline: December to May



Legend: ✓Completed ∼ Ongoing ☐ Pending

Implementation – System Architecture

- Lock-free Adaptive Radix Tree using multi-level nodes and CAS
- Mutex-based index for baseline comparison
- Parallel join with hot key detection and skewed data
- Epoch-based memory cleanup for lock-free execution
- Performance evaluation using throughput and latency

Design Rationale

Minimizes contention and handles skew in concurrent joins.

Testing Plan - Benchmarking Strategy

- Measure throughput for insert and find across Baseline, BW-Tree, ART, and Skew-Aware Join
- Report latency metrics (avg, P50, P95, P99) for each index
- Evaluate scalability using 1–16 threads and compute speedup
- Analyze skew sensitivity using Zipf workloads to study hot key behavior
- Verify correctness, contention, and memory overhead
- Perform warmup runs to avoid cold-cache effects

What Worked

- Baseline mutex-protected index compiles and runs correctly, achieving ∼27M ops/sec
- Lock-free ART index functions correctly for small datasets with ∼18M ops/sec
- Epoch-based memory reclamation enables safe cleanup in concurrent execution
- Benchmark framework successfully reports throughput and tail latency (P50, P95, P99)

Interpretation

The system establishes a stable and functional baseline for evaluating concurrent and lock-free indexing techniques.

Challenges and Learnings

- BW-Tree requires full non-leaf node traversal support beyond the current flat leaf implementation
- ART index traversal logic needs to be extended to handle deeper trees at larger scales
- Skew-aware join performance under high skew highlights the need for improved partitioning strategies
- Multi-threaded scalability experiments indicate areas for refinement in thread synchronization

Interpretation

These observations clearly identify the next steps for strengthening correctness, scalability, and skew resilience.

Limitations Identified

- BW-Tree lacks full non-leaf traversal support
- ART becomes unstable on larger datasets ($>$10K entries)
- Skew-aware join stalls under high data skew
- Multi-threaded scalability fails beyond one thread
- Lock-free ART shows higher latency than baseline

Remedies and Refinement Plan

- Implement full internal node routing in BW-Tree
- Complete deep traversal logic in ART
- Improve skew handling and partition logic
- Fix thread synchronization in epoch reclamation
- Optimize memory layout to reduce latency

Conclusion

- Demonstrates the feasibility of concurrency-aware indexing for distributed joins.
- Effectively addresses lock contention and data skew in controlled workloads.
- Requires further refinement for large-scale and highly concurrent execution.
- Ongoing improvements focus on achieving scalable, production-ready performance.

Thank You

Reference Implementation:
https://github.com/zakhaev26/db-research-paper